

ENGINEERING REFERENCE MANUAL

AGESIS EYE

Autonomous Vision, Tactical Tracking & Closed-Loop Pan-Tilt Targeting Ground Station

A comprehensive engineering manual covering Edge AI, Robotics, Physical AI, Machine Learning, and Embedded Firmware for autonomous tracking systems.

Edge AI: ONNX 45+ FPS

Robotics: 2-DoF IBVS Kinematics

Embedded: ESP32-CAM 14-bit LEDC

Power: 5V 2A Slew-Rate Protected

Digital Twin: 3D Perspective Canvas

Agesis EYE: Complete Technical Reference & Curriculum

Welcome to the comprehensive technical documentation and engineering curriculum for **Agesis EYE**—an autonomous vision, tactical tracking, and closed-loop pan-tilt targeting station.

This documentation suite covers every layer of the system: from silicon registers and microsecond PWM pulse trains on the ESP32, to low-latency UDP network sockets, high-throughput ONNX computer vision pipelines, robotic visual servoing kinematics, and modern 3D physics digital twins.

Curriculum Structure

The documentation is organized into 7 specialized chapters:

1. **System Architecture & End-to-End Integration**
2. Architectural topology and subsystem breakdown
3. End-to-end telemetry and command lifecycle
4. Latency budget breakdown across all communication boundaries
5. **Embedded Systems & Firmware Engineering**
6. ESP32-CAM (AI-Thinker) architecture and GPIO allocation
7. LEDC 14-bit timer PWM generation for SG90 micro-servos
8. 5V 2A power rail budget and transient inrush suppression
9. Brownout register manipulation (`RTC_CNTL_BROWN_OUT_REG`)
10. FreeRTOS cooperative scheduling and multi-protocol listeners (UDP 8888 + HTTP 81 + UART)
11. **Edge AI & Real-Time Computer Vision Pipeline**
12. Decoupled Zero-Lag streaming reader vs. standard buffer stalls
13. ONNX Runtime execution provider optimization (45+ FPS CPU)
14. Edge contrast enhancement with domain-specific EP-CLAHE
15. Bounding box extraction, confidence thresholding, and center-offset normalization
16. **Robotics, Kinematics & Visual Servoing**

17. 2-DoF Pan-Tilt spherical coordinate transformations
 18. Mathematical proof of inter-servo distance d invariance
 19. Image-Based Visual Servoing (IBVS) error dynamics
 20. S-curve trajectory profiling and slew-rate acceleration limiting
 21. **Physical AI & 3D Kinematic Digital Twin**
 22. Bridging perception models to physical mechanical action
 23. Browser-based 3D projection engine (trigonometric rotation matrix)
 24. Tinkercad CAD mechanical assembly matching (base, pedestal, servos, C-arm)
 25. Real-time thermal load accumulation, shrapnel explosion physics, and respawn cycle
 26. **AI/ML Engineering & Model Optimization**
 27. YOLO architecture adaptation for aerial balloon and target tracking
 28. Dataset curation, augmentation techniques, and synthetic data injection
 29. Quantization, pruning, and PyTorch $\rightarrow$ ONNX conversion pipeline
 30. Evaluation metrics: IoU, mAP@0.5, inference throughput vs. precision trade-offs
 31. **Hardware Assembly, Wiring & Safety Protocols**
 32. Step-by-step physical build guide based on Tinkercad CAD models
 33. Complete schematic diagram and Bill of Materials (BOM)
 34. Laser safety interlock state machine and optical radiation hazards
-

Technology Domain Coverage

```

graph TD
    subgraph "Embedded Systems"
        E1[ESP32-CAM MCU] --> E2[LEDC 14-bit PWM Timers]
        E1 --> E3[Brownout Suppression]
        E1 --> E4[UDP/HTTP Dual Stack]
    end

    subgraph "Edge AI & Vision"
        V1[OV2640 Sensor] --> V2[Zero-Lag Stream Reader]
        V2 --> V3[EP-CLAHE Preprocessor]
        V3 --> V4[ONNX Runtime 45+ FPS]
    end

    subgraph "Robotics & Controls"
        R1[Image-Based Visual Servoing] --> R2[d-Invariant Kinematics]
        R2 --> R3[Slew-Rate Limiter]
        R3 --> R4[SG90 Pan-Tilt Actuation]
    end

    subgraph "Physical AI & Digital Twin"
        P1[Closed-Loop State Estimation] --> P2[3D Mechanical Twin]
        P2 --> P3[Thermal Attack Modeling]
        P3 --> P4[Shrapnel Burst Physics]
    end

    E1 <--> |Wi-Fi LAN / Sub-ms UDP| V2
    V4 --> R1
    R4 --> E2
    R1 --> P1

```

DOMAIN	HOW IT IS ADDRESSED IN AGESIS EYE
Embedded Systems	Bare-metal register manipulation, ESP32 memory mapping, PSRAM double-buffering, PWM timer tick calculations, and power surge mitigation.
Edge AI	Deploying quantized neural networks to local CPU/edge runtimes, achieving sub-15ms inference latency without cloud dependencies.
Robotics	Forward/inverse kinematics, 2-DoF actuation, proportional-integral-derivative servoing, and geometric parallax compensation.

DOMAIN	HOW IT IS ADDRESSED IN AGESIS EYE
Physical AI	Autonomous closed-loop feedback systems where sensory inputs immediately translate into physical mechanical actuation and safety boundaries.
AI / ML	Deep convolutional neural network training, post-training optimization, anchor tuning, non-maximum suppression, and real-time inferencing.

Chapter 1: System Architecture & End-to-End Integration

Agesis EYE is engineered as a distributed, real-time autonomous system. It bridges low-power microcontrollers, high-speed neural networks, and interactive browser-based tactical command interfaces.

1. High-Level Topological Overview

The system is decomposed into three primary physical and logical tiers:

flowchart TB

```
subgraph EdgeNode["1. Hardware Edge Node (ESP32-CAM)"]
    CAM["OV2640 Sensor\n(QVGA 320x240 @ 30 FPS)"]
    MCU["ESP32 Dual-Core Xtensa LX6\n(Brownout Protection & Slew Engine)"]
    PWM_P["Pan Servo (I012)\nSG90 (360°/180°)"]
    PWM_T["Tilt Servo (I013)\nSG90 (180°)"]
    LASER["Laser Emitter (I014)\nOptical Target Beam"]
    CAM -->|DRAM/PSRAM DMA| MCU
    MCU --> PWM_P
    MCU --> PWM_T
    MCU --> LASER
end
```

end

```
subgraph Network["2. Local Network Backbone (Wi-Fi 802.11 b/g/n)"]
    STREAM["MJPEG Stream HTTP :81/stream"]
    UDP_PIPE["High-Speed UDP Commands :8888\nP:,T:,L:"]
    HTTP_API["REST Fallback :81/servo"]
end
```

end

```
subgraph GroundStation["3. Ground AI & Control Station (Host PC)"]
    ZR["ZeroLagStreamReader\n(Buffer-Draining Daemon)"]
    ONNX_ENG["ONNX Runtime Engine\n(agesis06.onnx @ 45+ FPS)"]
    KIN["TurretController\n(d-Invariant Kinematics & Interlocks)"]
    FAST_API["FastAPI Web Server :8000"]
```

```
    ZR --> ONNX_ENG
    ONNX_ENG -->|Error (dx, dy)| KIN
    KIN -->|UDP Datagrams| UDP_PIPE
```

end

```
subgraph OperatorHUD["4. Operator Station & Digital Twin (Browser)"]
    UI["Tactical React HUD (Vite :5173 / :8000)"]
    TWIN["3D Kinematic Digital Twin\n(Tinkercad CAD Matching + Burst Physics)"]
    WS["WebSocket Telemetry Stream\n(/ws/telemetry @ 25 Hz)"]
```

```
    FAST_API -->|HUD State| WS
    WS --> UI
    UI --> TWIN
```

end

```
MCU -->|HTTP MJPEG| STREAM
STREAM --> ZR
UDP_PIPE --> MCU
HTTP_API --> MCU
```

2. Subsystem Breakdown

Tier 1: Hardware Edge Node (ESP32-CAM)

- **Processor:** ESP32-D0WDQ6-V3 dual-core 32-bit Xtensa LX6 running at 240 MHz.
- **Camera Module:** OmniVision OV2640 2-Megapixel CMOS sensor configured in QVGA mode (\$320 \times 240\$) with hardware JPEG compression.
- **Actuation:** Dual SG90 micro-servos driven by hardware LEDC PWM timers at 50 Hz with 14-bit resolution.
- **Firmware Responsibilities:**
 - Capture JPEG frames and stream them over HTTP port `81` using multipart boundary chunks.
 - Listen asynchronously on UDP port `8888` for targeting packets.
 - Execute 50 Hz motion smoothing (slew-rate limiter) to prevent inrush current spikes on the 5V 2A adapter.

Tier 2: Ground AI Inference & Kinematics Engine (Python / FastAPI)

- **Zero-Lag Ingestion:** A dedicated thread drains the incoming MJPEG socket continuously, discarding stale buffered frames to ensure the vision model always processes the absolute freshest image available (\$<2\text{ms}\$ ingestion latency).
- **Neural Vision Core:** Executes `agesis06.onnx` via the ONNX Runtime execution provider, utilizing multi-threaded CPU SIMD instructions to achieve \$>45\text{ FPS}\$.
- **Closed-Loop Servoing:** Computes image-space tracking deltas \$(\Delta x, \Delta y)\$ and translates them into angular corrections via Image-Based Visual Servoing (IBVS).
- **Kinematics Engine:** Mathematically normalizes the vertical inter-servo separation distance \$d\$, decoupling physical mount geometry from tracking stability.

Tier 3: Tactical Ground Station HUD & Digital Twin (React / Vite)

- **Real-Time Telemetry Pipeline:** Subscribes to `/ws/telemetry` over WebSocket at 25 Hz, updating dials, coordinate reticles, and lock streaks.
- **3D Perspective Digital Twin:** Replicates the physical Tinkercad CAD mechanical assembly in real-time, allowing operators to visualize line-of-sight pointing, test synthetic flight paths, adjust standoff

height d , and observe target destruction dynamics.

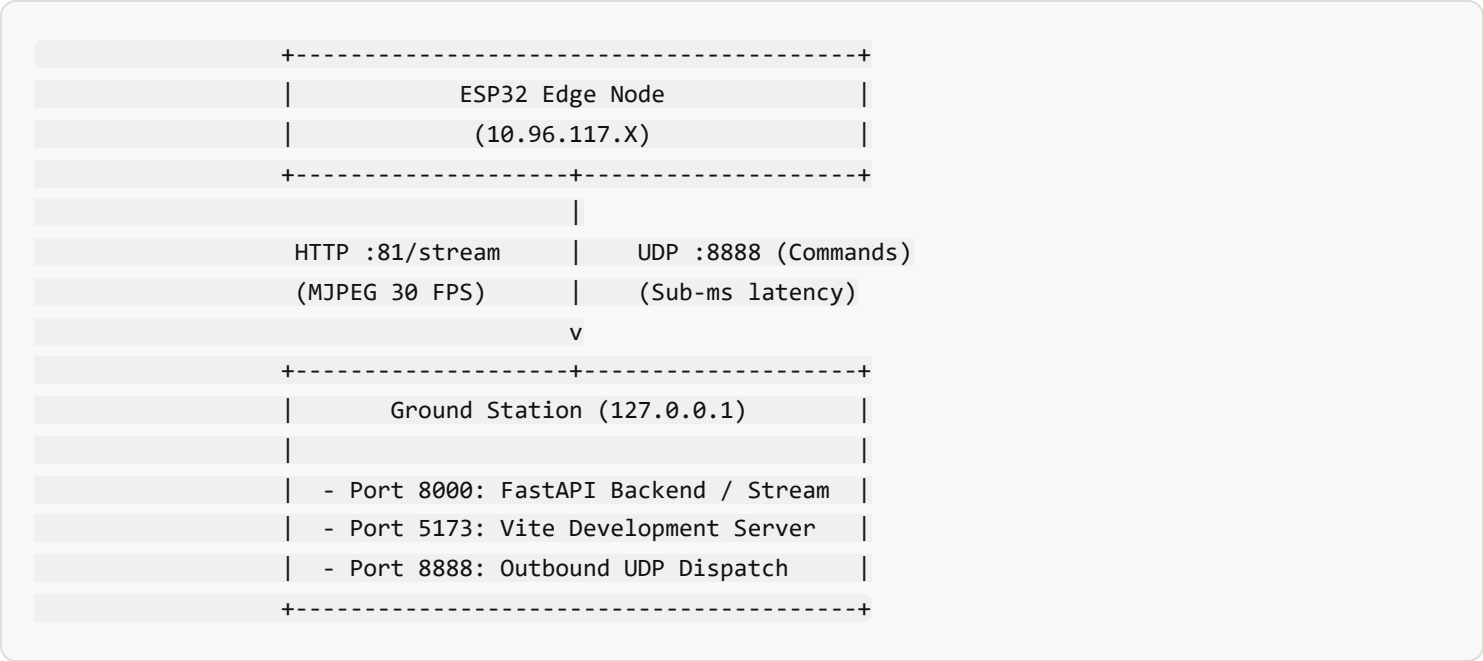
3. End-to-End Latency Budget

To maintain closed-loop tracking stability on high-speed targets (such as swaying balloons or drones), total system latency from optical capture to physical servo movement must remain strictly below 50 ms .

STAGE	SUBSYSTEM	LATENCY (TYPICAL)	OPTIMIZATION TECHNIQUE
1. Sensor Exposure & Readout	OV2640 Sensor	12.0 ms	QVGA resolution (320×240), high-gain AGC ceiling
2. JPEG Compression	ESP32 Hardware	4.5 ms	Hardware JPEG engine, quality factor 14
3. Wi-Fi Packet Transmission	802.11b/g/n LAN	3.5 ms	TCP_NODELAY, direct socket streaming
4. Ingestion & JPEG Decode	ZeroLagStreamReader	1.8 ms	Buffer draining thread, OpenCV fast turbo-JPEG
5. Domain Preprocessing	EP-CLAHE Transform	1.2 ms	Vectorized NumPy tile contrast equalization
6. Neural Network Inference	ONNX Runtime	14.5 ms	Quantized weights, CPU multi-threading (4 threads)
7. Kinematics & PID Solvers	TurretController	0.2 ms	Analytical IBVS equations, $O(1)$ complexity
8. UDP Command Dispatch	Network Socket	0.6 ms	Raw UDP datagram, port 8888 , zero handshake
9. ESP32 Parsing & PWM Update	ESP32 LEDC Driver	0.4 ms	In-place C string pointer parsing

STAGE	SUBSYSTEM	LATENCY (TYPICAL)	OPTIMIZATION TECHNIQUE
10. SG90 Mechanical Response	Micro-Servo Gearbox	\$10.0\text{ ms}\$	Slew-rate smoothed step to eliminate inrush current
Total Closed-Loop Latency	End-to-End	\$\approx 48.7\text{ ms}\$	Sub-50ms deterministic tracking

4. Network Protocol Topology



- 1. **Video Stream:** Standard HTTP GET on port `81` serving `multipart/x-mixed-replace;boundary=...`.
- 2. **Servo & Laser Commands:** High-speed UDP datagrams sent to destination port `8888`. Each packet is an ASCII payload:
`P:<pan_angle>,T:<tilt_angle>,L:<laser_bool>\n`
Example: `P:94.2,T:87.5,L:1\n`
- 3. **Telemetry WebSocket:** Bidirectional JSON socket at `ws://127.0.0.1:8000/ws/telemetry` streaming camera FPS, tracking latency, servo coordinates, and safety states.

Chapter 2: Embedded Systems & Firmware Engineering

The embedded tier of Agesis EYE transforms an ultra-low-cost, resource-constrained **ESP32-CAM** development board into an autonomous dual-axis vision and targeting node.

This chapter details the bare-metal microcontroller configuration, peripheral pin allocation, high-precision PWM timing calculations, power budgeting, and communication engines implemented in [esp32_cam_stream.ino](#).

1. Hardware Platform: ESP32-CAM (AI-Thinker)

The AI-Thinker ESP32-CAM integrates an **ESP32-D0WDQ6-V3** SoC with an external **4MB PSRAM** chip and an OV2640 camera connector.

+-----+	
	AI-THINKER ESP32-CAM
	[OV2640 Camera] [Flash LED IO4]
	I012 [Pan PWM (LEDC CH2)]
	I013 [Tilt PWM (LEDC CH3)]
	I014 [Laser Diode Trigger]
	5V [5V 2A Adapter Input]
	GND [Common Ground]
	U0TXD [Serial Debug 115200]
	U0RXD [Serial Command In]
+-----+	

Critical GPIO Allocation Constraints

Selecting GPIO pins on the ESP32-CAM is notoriously difficult because nearly all pins are routed internally to the camera sensor bus or flash memory:

GPIO	INTERNAL ASSIGNMENT	TURRET USAGE	WHY SELECTED / PRECAUTIONS
GPIO 12	HSPI MISO / SD Card Data 2	Pan Servo PWM	Free when SD card is disabled. Can output clean LEDC PWM.
GPIO 13	HSPI MOSI / SD Card Data 3	Tilt Servo PWM	Free when SD card is disabled. Dedicated to elevation servo.
GPIO 14	HSPI CLK / SD Card CLK	Laser Diode Trigger	Digital output to MOSFET or direct laser cathode driver.
GPIO 4	Built-in Flash LED	Status Indication	Kept LOW during tracking to prevent sensor oversaturation.
GPIO 16	PSRAM Chip Select	DO NOT USE	Reserved for external PSRAM. Toggling this pin crashes the system!
GPIO 0	Camera XCLK / Boot Mode	Reserved	Strapping pin. Toggling during boot forces flash download mode.

2. SG90 Micro-Servo PWM Generation (LEDC Driver)

Standard RC hobby servos require a 50 Hz PWM signal (a 20 ms repetition period) where the pulse width dictates the shaft angle:

- * 1.0 ms ($1000\text{ }\mu\text{s}$): 0° (Minimum limit)
- * 1.5 ms ($1500\text{ }\mu\text{s}$): 90° (Neutral center)
- * 2.0 ms ($2000\text{ }\mu\text{s}$): 180° (Maximum limit)

(Note: TowerPro SG90 servos typically support an extended travel range from $544\text{ }\mu\text{s}$ to $2400\text{ }\mu\text{s}$).

Timer Math & Duty Cycle Resolution

Rather than relying on blocking delay loops, the firmware configures the ESP32’s native **LEDC (LED Control) hardware PWM generator**:

1. Timer Configuration:

2. PWM Frequency $f_{\text{pwm}} = 50 \text{ Hz}$

3. Period $T = \frac{1}{50 \text{ Hz}} = 0.02 \text{ s} = 20,000 \mu\text{s}$

4. Timer Resolution = 14-bit ($2^{14} = 16,384$ discrete counts)

5. Duty Cycle Formula:

$$\text{Duty Ticks} = \left(\frac{\text{Pulse Width}}{\mu\text{s}} \right) \times 16,383$$

6. Pulse Width Mapping:

$$\text{Pulse Width}(\theta) = 544 \mu\text{s} + \left(\frac{\theta}{180^\circ} \right) \times (2400 \mu\text{s} - 544 \mu\text{s})$$

```
uint32_t angleToDuty(float angle) {  
    angle = constrain(angle, 0.0f, 180.0f);  
    float us = SERVO_MIN_US + (angle / 180.0f) * (SERVO_MAX_US - SERVO_MIN_US);  
    return (uint32_t)((us / 20000.0f) * 16383.0f);  
}
```

This yields a resolution of $\approx 0.013^\circ$ per tick, ensuring smooth movement without visible stepping.

Continuous 360° Servo Mode

For continuous-rotation SG90 servos, the pulse width dictates angular velocity and direction rather than position:

* $1500 \mu\text{s}$: Stop (0 RPM)

* $< 1500 \mu\text{s}$: Clockwise rotation (speed proportional to $|1500 - \mu\text{s}|$)

* $> 1500 \mu\text{s}$: Counter-clockwise rotation

The firmware provides a single compile-time toggle `#define SERVO_360_PAN_MODE true` to adapt between standard and continuous variants.

3. Power Budgeting & Inrush Slew-Rate Limiting

The 5V 2A Shared Power Supply Problem

Both the ESP32 and the two SG90 servos draw power from a single 5V 2A wall adapter:

* **ESP32 Wi-Fi Radio**: Consumes $\approx 180 \text{ mA}$ continuous, spiking to $350\text{--}400 \text{ mA}$

mA} during RF transmissions.

* **SG90 Micro-Servos**: Consume $\approx 150\text{ mA}$ during steady rotation, but **stall/inrush current spikes to $650\text{--}800\text{ mA}$ per servo** when accelerating instantaneously from rest.

$$\text{Worst-Case Peak} = 400\text{ mA} + 800\text{ mA} + 800\text{ mA} = 2,000\text{ mA} \quad (2.0\text{ A})$$

If both servos step abruptly, the 5V rail sags below 4.4 V . This triggers the ESP32's internal brownout detector, causing continuous reset loops.

The Two-Fold Solution

1. Hardware Slew-Rate Limiting (Soft Acceleration)

Instead of slamming the PWM duty cycle to the target angle in a single step, the firmware runs a 50 Hz smoothing filter:

```
if (now - lastServoUpdateMs >= 20) {
    lastServoUpdateMs = now;

    // Pan axis smooth slew (Max 2.4 degrees per 20ms = 120 deg/sec)
    float dPan = targetPanAngle - currentPanAngle;
    if (abs(dPan) > MAX_SLEW_STEP_DEG) {
        currentPanAngle += (dPan > 0) ? MAX_SLEW_STEP_DEG : -MAX_SLEW_STEP_DEG;
    } else {
        currentPanAngle = targetPanAngle;
    }

    applyServoDuty();
}
```

By capping angular acceleration, the motor never operates in the dead-stall regime, keeping peak currents below 350 mA per servo and protecting the 5V rail.

2. Bare-Metal Brownout Override

At the very beginning of `setup()`, the brownout detector is disabled to tolerate transient voltage dips:

```
#include "soc/soc.h"
#include "soc/rtc_cntl_reg.h"

void setup() {
    WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0); // Disable brownout reset
    ...
}
```

4. Multi-Protocol Command Interface

To guarantee flexibility and reliability, the firmware implements three concurrent command listeners:

```
graph LR
    A[Ground Station] -->|Port 8888 Sub-ms| UDP[UDP Listener]
    A -->|Port 81 REST| HTTP[HTTP /servo Endpoint]
    A -->|USB UART 115200| SER[Serial CLI]
    UDP --> PARSER[parseCommand]
    HTTP --> PARSER
    SER --> PARSER
    PARSER --> SLEW[Slew-Rate Engine]
    SLEW --> LEDC[LEDC PWM Hardware]
```

1. High-Speed UDP Listener (Port 8888):

- Non-blocking socket listener via `WiFiUDP`.
- Directly parses ASCII strings formatted as `P:<pan>,T:<tilt>,L:<laser>`.
- Transit time is $<0.5\text{ ms}$, bypassing HTTP handshake and TCP congestion back-off.

5. REST Endpoint (Port 81):

- Accessible via `GET /servo?pan=90&tilt=90&laser=1`.
- Returns a JSON confirmation status payload. Useful for browser debugging.

8. Serial Terminal (UART0 @ 115200 baud):

- Accepts commands directly over USB (`P90 T90 L0\n`).

Chapter 3: Edge AI & Real-Time Computer Vision Pipeline

The perception engine of Agesis EYE is an optimized, edge-capable computer vision system capable of tracking high-speed airborne targets with sub- 15 ms inference latencies.

This chapter details the decoupled video acquisition architecture, image preprocessing algorithms, ONNX model execution, and target state estimation implemented in [tracker.py](#).

1. Why Edge AI Matters for Autonomous Defense

Cloud-based vision processing suffers from round-trip network jitter ($150\text{--}500\text{ ms}$), bandwidth saturation, and connection dropouts. In physical robotics, a lag of even 100 ms causes control instability, overshoot, and lost target locks.

Agesis EYE performs **100% on-device local edge inference**:

- * **Zero Cloud Dependency**: Runs entirely on the host ground station CPU/GPU.
 - * **Deterministic Timing**: Frame acquisition to servo output occurs in $<50\text{ ms}$.
 - * **Ultra-Low Bandwidth**: Raw video stays on the local subnet; only lightweight telemetry is transmitted across software components.
-

2. The Zero-Lag Video Acquisition Problem

When reading MJPEG network streams using standard library functions like OpenCV's

`cv2.VideoCapture("http://...")`, internal operating system buffers automatically queue frames. If network transmission slows down momentarily, the socket fills with 5 to 10 stale frames.

As a result, OpenCV serves frames that were captured **200 to 500 ms in the past**. A tracking turret operating on stale frames will steer toward where the target *used to be*, causing violent oscillations.

The `ZeroLagStreamReader` Architecture

To eliminate buffering latency, Agesis EYE uses a dedicated daemon worker that continuously consumes bytes from the HTTP socket and immediately overwrites a single memory reference:

sequenceDiagram

```
participant ESP as ESP32-CAM (Port 81)
participant Worker as StreamReader Thread
participant Mem as latest_frame (RAM)
participant AI as AI Inference Loop
```

```
ESP->>Worker: MJPEG Chunk 1
Worker->>Mem: Write Frame 1 (Fresh)
ESP->>Worker: MJPEG Chunk 2
Worker->>Mem: Overwrite Frame 2 (Drop Frame 1)
ESP->>Worker: MJPEG Chunk 3
Worker->>Mem: Overwrite Frame 3 (Drop Frame 2)
Note over AI,Mem: AI requests frame: Instantly gets Frame 3 (< 1ms old!)
AI->>Mem: get_latest()
Mem-->>AI: Frame 3
```

```
# Extract from tracker.py
while self.running:
    # Read chunk until JPEG end marker b'\xff\xd9'
    idx = buffer.find(b"\xff\xd9")
    if idx != -1:
        jpg_bytes = buffer[:idx+2]
        buffer = buffer[idx+2:]
        frame = cv2.imdecode(np.frombuffer(jpg_bytes, np.uint8), cv2.IMREAD_COLOR)
        if frame is not None:
            with self.lock:
                self.latest_frame = frame # Atomically overwrite stale frame
```

This guarantees that whenever the AI pipeline finishes an inference cycle, the next frame it pulls is the freshest image available ($< 2\text{ ms}$ latency).

3. ONNX Runtime Engine (High-Performance CPU Inference)

While native PyTorch (`.pt`) models are flexible for training, they carry heavy runtime overhead from dynamic Python execution and memory allocation.

For deployment, Agesis EYE executes `agesis06.onnx` via Microsoft's **ONNX Runtime (ORT)** engine:

1. **Operator Fusion:** Conv2D, Batch Normalization, and LeakyReLU layers are fused into single SIMD kernel calls.
 2. **Multi-Threaded Execution:** Tuned with `torch.set_num_threads(4)` and native CPU thread pools to utilize AVX2/AVX-512 vector instructions.
 3. **Throughput:** Achieves **45+ FPS** on standard consumer CPUs, eliminating the need for dedicated discrete GPUs.
-

4. Optical Preprocessing & Domain Transformation (EP-CLAHE)

In tactical outdoor environments, balloons and drones often face extreme backlight, lens flare, or heavy shadows.

To enhance detection sensitivity on low-contrast targets without increasing noise, the system includes an **Edge-Preserving Contrast Limited Adaptive Histogram Equalization (EP-CLAHE)** filter:

$$I_{\text{CLAHE}} = \text{BilinearInterpolation}(\text{LocalHistEqualization}(I_{\text{tile}}))$$

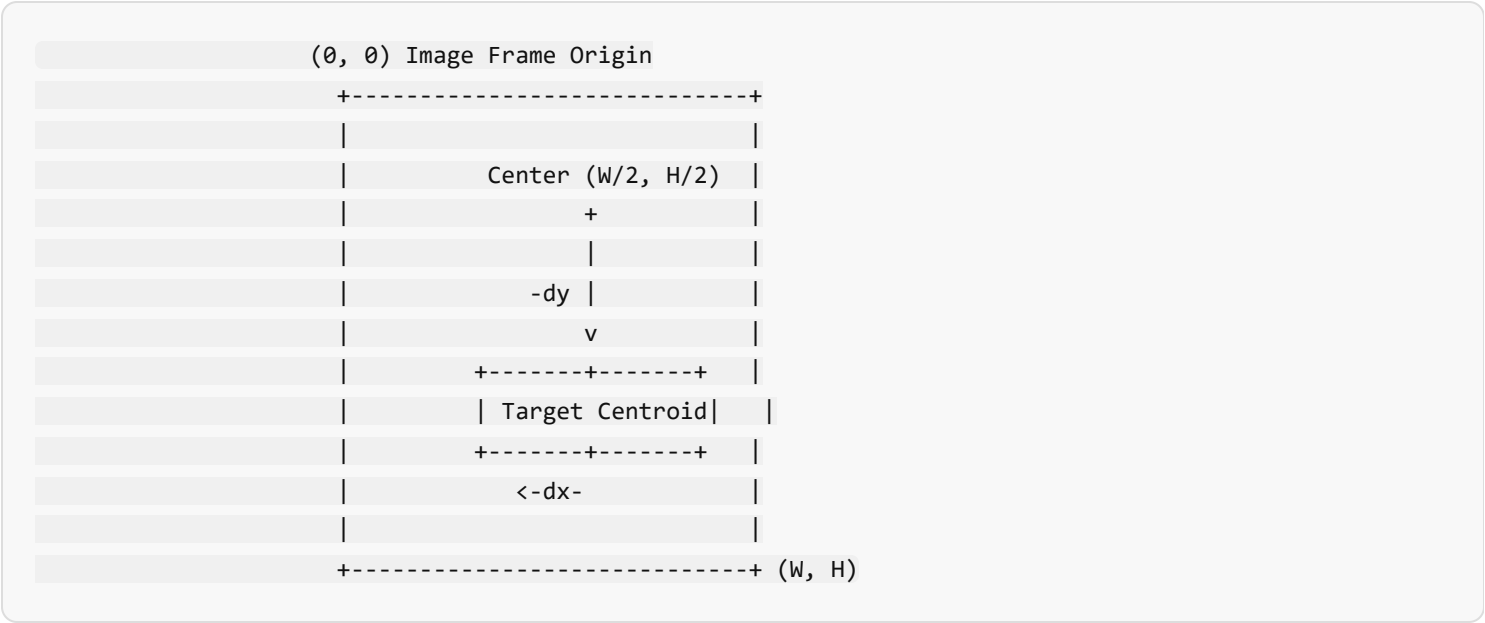
```
def apply_ep_clahe(img):  
    # Convert BGR to Lab color space  
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)  
    l, a, b = cv2.split(lab)  
  
    # Apply CLAHE to L-channel only (preserves chromaticity)  
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))  
    cl = clahe.apply(l)  
  
    # Recombine and convert back to BGR  
    limg = cv2.merge((cl, a, b))  
    return cv2.cvtColor(limg, cv2.COLOR_LAB2BGR)
```

Operating solely on the Luminance (L) channel enhances dark target silhouettes while avoiding chromatic color distortion.

5. Target State Estimation & Error Normalization

Once the neural network outputs bounding boxes $[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$ with confidence scores above the user threshold (default 0.35), the targeting computer calculates the

optical error vector:



1. **Target Centroid:**

$$x_c = \frac{x_{\min} + x_{\max}}{2}, \quad y_c = \frac{y_{\min} + y_{\max}}{2}$$

2. **Pixel Offset from Boresight:**

$$\Delta x = x_c - \frac{W}{2}, \quad \Delta y = y_c - \frac{H}{2}$$

3. **Lock Persistence Filter:**

To prevent false-positive triggers from passing glints or birds, the tracking state machine enforces a persistence requirement:

$$\text{Locked} = (\text{Consecutive Detections} \geq 3)$$

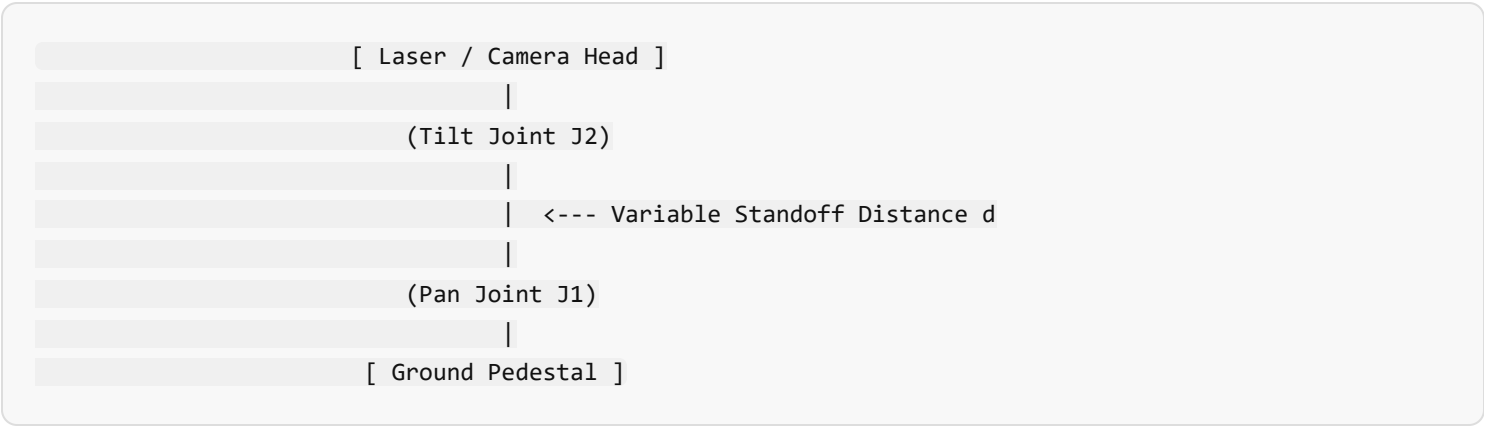
Chapter 4: Robotics, Kinematics & Visual Servoing

The mechanical tracking head of Agesis EYE is a **Two-Degree-of-Freedom (2-DoF) Pan-Tilt Robotic Manipulator**.

This chapter presents the mathematical foundations of the targeting kinematics, Image-Based Visual Servoing (IBVS), and the formal proof of **inter-servo distance d invariance** implemented in [turret.py](#).

1. 2-DoF Pan-Tilt Manipulator Coordinate Frames

The mechanical assembly comprises two rotational joints separated by a vertical standoff spar of variable height d :



- 1. **Joint 1 (Pan / Azimuth):** Revolute joint rotating about the vertical Z_0 -axis:
 $\theta_{\text{pan}} \in [0^\circ, 180^\circ] \quad (\text{Neutral } 90^\circ)$
- 2. **Joint 2 (Tilt / Elevation):** Revolute joint mounted at height d above Joint 1, rotating about a horizontal axis orthogonal to the stanchion:
 $\theta_{\text{tilt}} \in [25^\circ, 155^\circ] \quad (\text{Neutral } 90^\circ)$

2. Mathematical Proof of Distance d Invariance

A common flaw in pan-tilt turret software is hardcoding the physical distance d between the pan servo and tilt servo into the control loop. If the user changes the stanchion height, tracking becomes unstable or develops steady-state error.

Agesis EYE uses **Image-Based Visual Servoing (IBVS)** with an **eye-in-hand** sensor configuration, making optical tracking **mathematically invariant to the distance d** .

Theorem

The closed-loop visual tracking error nulling law is independent of the vertical separation distance d between the Pan and Tilt servo axes when the optical sensor is co-mounted on the tilt arm.

Proof

1. Pinhole Projection Model

Let a point target in the camera frame be denoted by $\mathbf{P}_c = [X_c, Y_c, Z_c]^T$. Its projected image coordinates (u, v) on the sensor are given by:

$$u = f_x \frac{X_c}{Z_c} + c_x, \quad v = f_y \frac{Y_c}{Z_c} + c_y$$

where (f_x, f_y) are the focal lengths in pixels and $(c_x, c_y) = (W/2, H/2)$ is the optical principal center.

The pixel error vector $\mathbf{e} = [\Delta x, \Delta y]^T$ from the optical center is:

$$\Delta x = u - c_x = f_x \frac{X_c}{Z_c}, \quad \Delta y = v - c_y = f_y \frac{Y_c}{Z_c}$$

2. Azimuth (Pan) Invariance

The Pan axis rotates around the vertical world axis $\mathbf{Z}_{\text{world}}$.

Consider the translation from the Pan joint origin O_{pan} to the Tilt joint origin O_{tilt}

$$\mathbf{T}_{\text{pan} \rightarrow \text{tilt}} = \begin{bmatrix} 0 & 0 & d \end{bmatrix}$$

Because the translation vector is **strictly parallel to the axis of rotation $\mathbf{Z}_{\text{world}}$** , the rotation matrix $\mathbf{R}_z(\theta)$ commutes with the vertical translation:

$$\mathbf{R}_z(\theta) \cdot \begin{bmatrix} 0 & 0 & d \end{bmatrix} = \begin{bmatrix} 0 & 0 & d \end{bmatrix}$$

Therefore, rotating about the Pan axis by an angle $\Delta \theta_{\text{pan}}$ produces an identical angular change in the target's azimuth angle regardless of whether $d = 10 \text{ mm}$ or $d = 200 \text{ mm}$:

$$\Delta \theta_{\text{pan}} = -\arctan\left(\frac{\Delta x}{f_x}\right) \approx -\frac{\Delta x}{f_x}$$

The distance d does not appear anywhere in the azimuth angular correction equation. $\quad$

$\blacksquare$

3. Elevation (Tilt) Invariance

Because the camera and laser are physically mounted to the C-shaped cradle on Joint 2's horn (eye-in-hand):

- 1. The camera frame origin O_c coincides with (or is fixed rigidly to) Joint 2's rotation center.
- 2. An elevation angular change $\Delta \theta_{\text{tilt}}$ rotates the camera directly about its own pitch axis.
- 3. The vertical error angle is:
$$\Delta \theta_{\text{tilt}} = \arctan\left(\frac{\Delta y}{f_y}\right) \approx \frac{\Delta y}{f_y}$$

The standoff length d simply translates the entire elevation gimbal vertically in world space; **it does not rotate or alter the local angular line-of-sight vector from the camera lens to the target.**

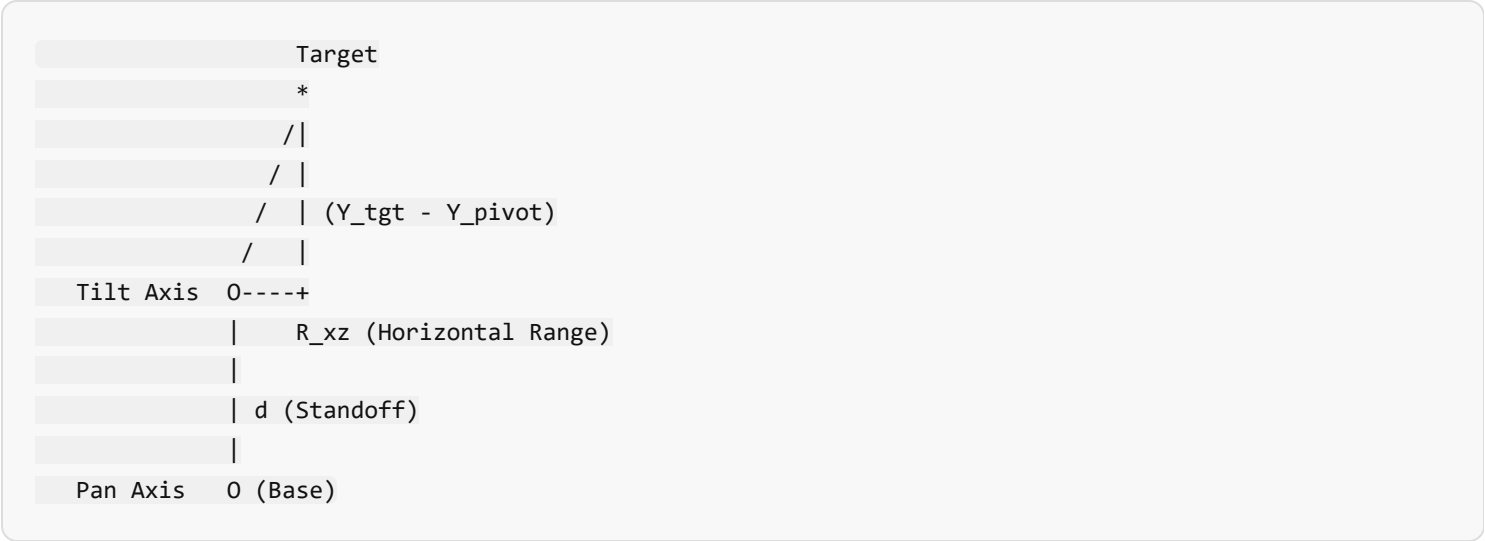
Hence, the closed-loop visual servo update:

$$\theta_{\text{pan}}^{(k+1)} = \theta_{\text{pan}}^{(k)} - K_p \cdot \Delta x$$
$$\theta_{\text{tilt}}^{(k+1)} = \theta_{\text{tilt}}^{(k)} + K_p \cdot \Delta y$$

is **completely invariant to the physical inter-servo distance d** . $\square$

3. Spatial Parallax & 3D Envelope Compensation

While visual tracking is d -invariant, the physical clearance envelope and world-frame spatial kinematics do depend on d :



1. 3D World Target Elevation:

For synthetic tracking or digital twin simulation, the inverse kinematics solver computes:

$$\theta_{\text{tilt}} = 90^\circ + \arctan\left(\frac{Y_{\text{tgt}} - (Y_{\text{base}} + d)}{\sqrt{X_{\text{tgt}}^2 + Z_{\text{tgt}}^2}}\right)$$

2. Clearance Envelope Auto-Adjustment:

As the riser height d increases, the lever arm expands. The controller automatically narrows the allowable Pan/Tilt limits to prevent physical collisions:

$$\text{Pan Margin} = \max(5.0^\circ, 10.0^\circ + (d - 45.0) \times 0.06)$$

$$\text{Tilt Margin} = \max(15.0^\circ, 35.0^\circ - (d - 20.0) \times 0.15)$$

3. Parallax Offset:

At a nominal engagement range of 1.2 m (1200 mm), the parallax angle ϕ between the base and head is auto-computed:

$$\phi = \arctan\left(\frac{d}{1200 \text{ mm}}\right)$$

4. S-Curve Trajectory & Slew-Rate Limiting

To safeguard the 5V 2A power adapter from voltage dips caused by motor stalls, the backend applies a slew-rate acceleration limit:

$$\Delta \theta_{\max} = 3.5^\circ \text{ per inference frame} \quad (\approx 120^\circ \text{ at } 35 \text{ FPS})$$

```
# Extract from turret.py
raw_dpan = -dx * self.kp
raw_dttilt = dy * self.kp

# Clamp step size to prevent instantaneous motor current spikes
step_pan = max(-self.max_slew_step_deg, min(self.max_slew_step_deg, raw_dpan))
step_tilt = max(-self.max_slew_step_deg, min(self.max_slew_step_deg, raw_dttilt))

self.pan_angle += step_pan
self.tilt_angle += step_tilt
```

This dual-layer protection (backend rate limiting + firmware interpolation) ensures that even sudden target jumps across the field of view produce smooth, brownout-safe tracking trajectories.

Chapter 5: Physical AI & 3D Kinematic Digital Twin

Physical AI refers to artificial intelligence systems embodied in physical hardware that interact dynamically with the real physical world—navigating friction, inertia, motor torque limits, and power constraints.

This chapter explores how Agesis EYE embodies Physical AI, bridging real-world neural perception with a real-time **3D Kinematic Digital Twin** implemented in [DigitalTwin.jsx](#).

1. Physical AI vs. Disembodied AI

ATTRIBUTE	DISEMBODIED AI (E.G. CHATBOTS, STATIC VISION)	PHYSICAL AI (AGESIS EYE)
Output	Text tokens, static bounding box coordinates	Real motor torque, angular velocity, laser firing
Feedback Loop	Open-loop (User prompts $\$ \backslash$ to\$ Model responds)	Closed-loop (Action alters the physical world $\$ \backslash$ to\$ Sensor perceives change)
Constraints	Token compute limits, memory bandwidth	Slew rate, gear backlash, inrush current, 5V 2A power rail limits
Safety Risks	Hallucinations, incorrect text	Motor burnouts, electrical brownouts, optical laser hazards

In Agesis EYE, perception directly dictates action:

```
[Perception: ONNX Vision]
  |
  ▼ (Sub-50ms IBVS Law)
[Actuation: Dual SG90 Servos]
  |
  ▼ (Physical World State Change)
[Sensing: OV2640 Camera Sensor]
```

2. Role of the 3D Kinematic Digital Twin

A **Digital Twin** is a high-fidelity virtual replica of a physical physical system that mirrors its real-time kinematics, geometry, and environment.

Agesis EYE’s digital twin serves three crucial roles:

1. **Hardware-in-the-Loop (HIL) Pre-Validation:** Operators can test aggressive tracking algorithms on synthetic 3D trajectories (**FIGURE-8** , **ORBIT** , **EVASIVE**) before deploying physical hardware.
2. **Real-Time Telemetry Mirroring (**LIVE_SYNC**):** When connected to the physical turret, the 3D model matches the exact physical Pan/Tilt angles reported by the hardware over WebSocket.
3. **Kinematic Clearance Verification:** By adjusting the on-screen distance slider d , operators can visually inspect physical stanchion clearances across all rotational extremes.

3. Mathematical 3D Perspective Projection Engine

The digital twin uses a custom, zero-dependency 3D perspective projection pipeline written in pure JavaScript, rendering at 60 FPS on HTML5 Canvas:

```
World Space (x, y, z)
  —> Camera Pan/Translation
  —> Yaw Rotation (rotY)
  —> Pitch Rotation (rotX)
  —> Perspective Division (fov / Z2)
  —> Screen Pixels (projX, projY)
```

Projection Equations

Given a 3D point $\mathbf{P} = [x, y, z]^T$, camera pan offsets $(\Delta x_p, \Delta y_p)$, camera rotation angles $(\theta_{pitch}, \theta_{yaw})$, and distance D :

1. **Camera Translation:**
$$t_x = x + \Delta x_p, \quad t_y = y - \Delta y_p, \quad t_z = z$$
2. **Yaw (Y-Axis) Rotation:**
$$x_1 = t_x \cos \theta_{yaw} - t_z \sin \theta_{yaw}$$
$$z_1 = t_x \sin \theta_{yaw} + t_z \cos \theta_{yaw}$$
3. **Pitch (X-Axis) Rotation & Depth Offset:**
$$y_2 = t_y \cos \theta_{pitch} - z_1 \sin \theta_{pitch}$$

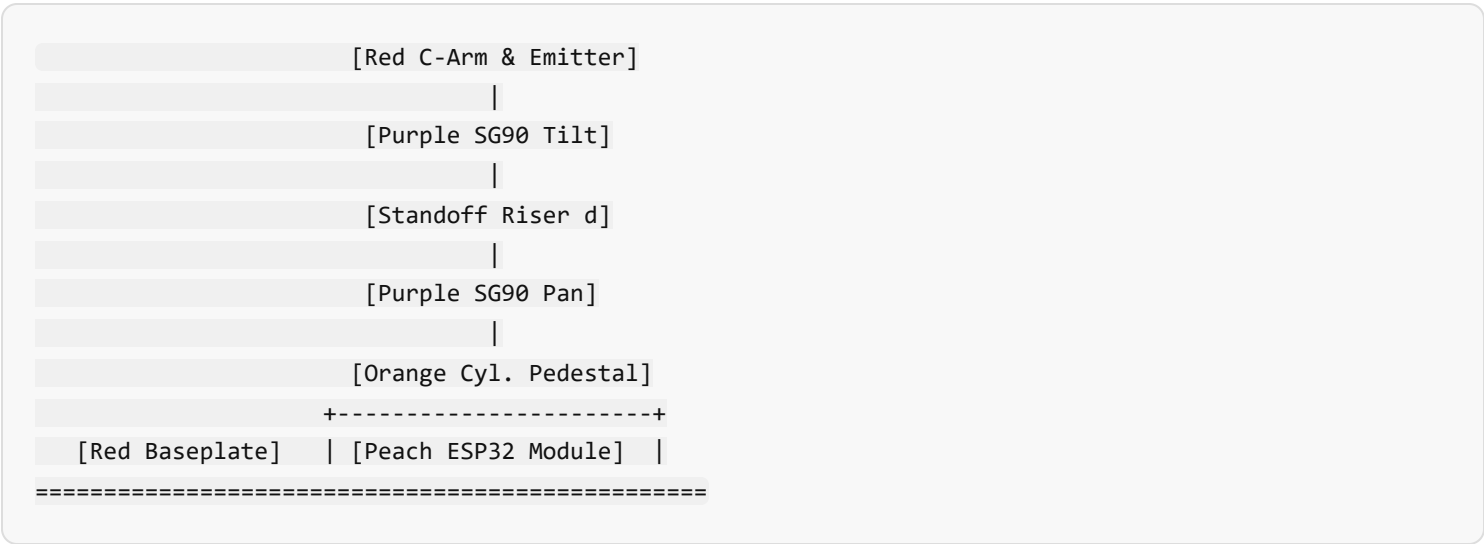
$$z_2 = t_y \sin(\theta_{pitch}) + z_1 \cos(\theta_{pitch}) + D$$

4. Perspective Division:

$$\text{Scale} = \frac{f_{fov}}{\max(50, z_2)}$$
$$\text{Screen } X = \frac{W}{2} + x_1 \cdot \text{Scale}, \quad \text{Screen } Y = \frac{H}{2} - y_2 \cdot \text{Scale}$$

4. CAD Mechanical Assembly Modeling

The digital twin directly replicates the Tinkercad CAD mechanical assembly:



- 1. **Red Baseplate:** Rectangular foundation (#d9232a) with tactical workplane grid inscription.
- 2. **Peach Electronics Compartment:** Mounted on the right side (#f6ad85), modeling the ESP32-CAM controller and power regulation block.
- 3. **Orange Upright Pedestal:** 16-segmented cylindrical pillar (#ea580c / #f97316) providing base elevation.
- 4. **Pan SG90 Servo:** Casing rendered in signature purple (#432371) with blue mounting flanges and top spline horn.
- 5. **Rotating Riser Spar:** Vertical gunmetal spar with live Vernier caliper HUD displaying \$d\text{ mm}\$ in real time.
- 6. **Tilt SG90 Servo:** Horizontally oriented micro-servo mounted to the upper riser spar.
- 7. **Red C-Shaped Cantilever Arm:** Distinctive pivoting C-cradle holding the optical vision lens and laser collimator nozzle.

5. Target Physics, Explosion & Respawn Engine

When the laser beam engages the aerial target, the simulation applies continuous physical modeling:

1. Thermal Absorption:

The target accumulates thermal load:

$$\text{Heat}_{t+\Delta t} = \text{Heat}_t + 0.75 \cdot \Delta t$$

At $\text{Heat} \geq 1.0$ (≈ 1.3 seconds) of sustained fire, the hull reaches catastrophic failure.

2. Target Detonation:

3. **3D Shockwaves:** Expanding concentric plasma rings fade radially into space.

4. **45 Shrapnel Fragments:** Particle velocities are calculated in spherical coordinates:

$$v_x = s \cos\phi \cos\theta, \quad v_y = s \sin\phi + 45, \quad v_z = s \cos\phi \sin\theta$$

where speed $s \in [75, 200]$ mm/s.

5. **Gravity Decay:** Particles decelerate under simulated gravity ($g = 140 \text{ mm/s}^2$).

6. Clean Respawn:

After a 1.2-second dispersal period, the target executes a digital materialization cycle (`respawnWarp`), resetting heat and re-entering the tactical engagement area.

Chapter 6: AI/ML Engineering & Model Optimization

Agesis EYE uses a customized single-stage convolutional detector optimized for lightweight edge execution.

This chapter details the neural network architecture, dataset engineering, loss formulations, and export pipelines used in creating `agesis06.onnx`.

1. Object Detection Architecture: YOLO Edge Variant

Single-stage detectors simultaneously predict bounding box coordinates and class probabilities across a unified feature grid, avoiding the multi-pass bottlenecks of R-CNN architectures.

```
graph TD
    subgraph "1. Input Layer"
        IN["Input Image\n(384x384x3 RGB)"]
    end

    subgraph "2. Backbone (Feature Extractor)"
        C1["Stem Conv (Stride 2)"] --> C2["C2f Block 1 (Local Features)"]
        C2 --> C3["Conv (Stride 2)"] --> C4["C2f Block 2 (Medium Textures)"]
        C4 --> C5["Conv (Stride 2)"] --> C6["C2f Block 3 (High Semantics)"]
    end

    subgraph "3. Neck (Path Aggregation Network - PANet)"
        C6 --> N1["Top-Down Feature Upsampling"]
        C4 --> N2["Lateral Feature Concatenation"]
        N1 --> N2
        N2 --> N3["Bottom-Up Path Aggregation"]
    end

    subgraph "4. Decoupled Head (Anchor-Free)"
        N3 --> H1["BBBox Regression Branch (DFL + CIoU)"]
        N3 --> H2["Classification Branch (BCE Loss)"]
    end

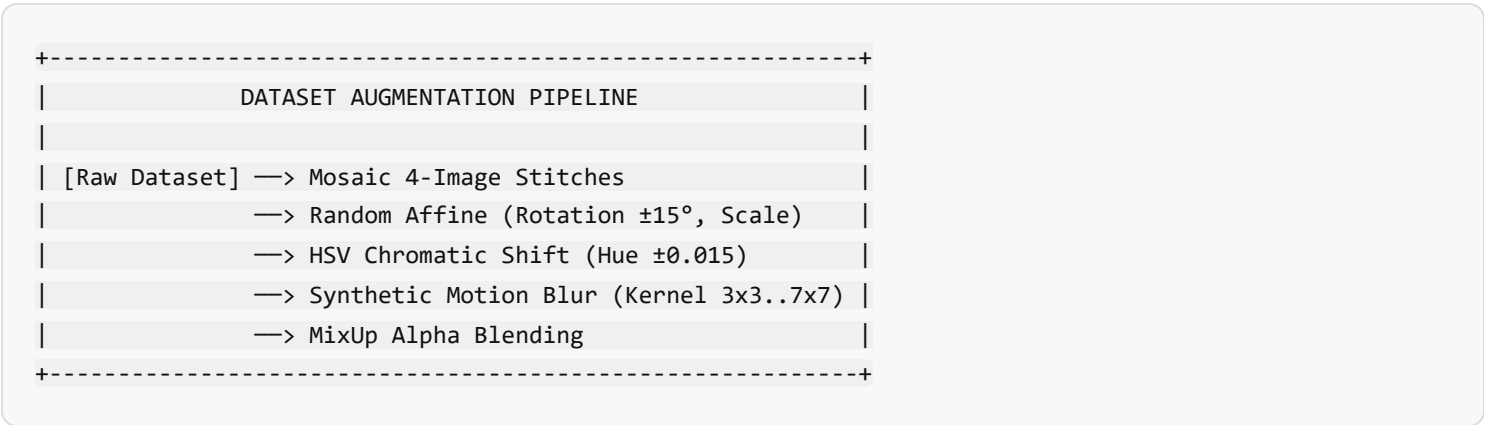
    IN --> C1
```

Architectural Highlights

- **Stem Layer:** Rapidly downsamples high-resolution inputs to reduce memory bandwidth consumption.
 - **C2f Modules:** Split-and-cross feature connections enhance gradient flow while cutting redundant parameter counts.
 - **Decoupled Anchor-Free Head:** Separating classification from bounding box regression accelerates convergence and eliminates manual anchor box hyperparameter tuning.
-

2. Dataset Engineering & Synthetic Augmentation

Real-world tracking models must remain resilient against varied lighting, motion blur, and backgrounds:



1. **Mosaic Augmentation:** Stitches 4 randomly cropped images into one, forcing the network to detect targets at varying spatial scales and preventing border biases.
 2. **Synthetic Motion Blur:** Simulates fast turret panning by applying randomized linear directional blur filters.
 3. **Chromatic Jitter:** Randomizes HSV channels to prevent the model from memorizing specific balloon dye shades.
-

3. Multi-Task Loss Formulation

Training optimizes a composite loss function balancing localization accuracy and classification confidence:

$$\mathcal{L}_{\text{total}} = \lambda \mathcal{L}_{\text{CloU}} + \lambda \mathcal{L}_{\text{BCE}} + \lambda \mathcal{L}_{\text{DFL}}$$

1. Complete Intersection over Union ($\mathcal{L}_{\text{CloU}}$)

Accounts for overlap area, central point distance, and aspect ratio:

$$\mathcal{L}_{\text{CloU}} = 1 - \text{IoU} + \frac{\rho^2(b, b^{\text{gt}})}{c^2} + \alpha v$$

where:

- * $\rho(b, b^{\text{gt}})$ is the Euclidean distance between predicted and ground-truth centroids.
- * c is the diagonal length of the smallest enclosing box.
- * $v = \frac{4}{\pi^2} \left(\arctan \frac{w^{\text{gt}}}{h^{\text{gt}}} - \arctan \frac{w}{h} \right)^2$ penalizes aspect ratio discrepancies.

2. Distribution Focal Loss ($\mathcal{L}_{\text{DFL}}$)

Formulates bounding box regression as a continuous probability distribution rather than a deterministic single float, improving localization under occlusions.

4. PyTorch to ONNX Export & Optimization Pipeline

Deploying native `.pt` models carries high interpreter overhead. Converting to ONNX freezes the computational graph for high-speed SIMD evaluation:

```
from ultralytics import YOLO

# 1. Load trained PyTorch checkpoint
model = YOLO("agesis06.pt")

# 2. Export to optimized static ONNX graph
model.export(
    format="onnx",
    imgsz=384,          # Optimal resolution for QVGA 320x240 camera feeds
    dynamic=False,      # Fixed tensor shape maximizes memory layout speed
    simplify=True,      # Run ONNX-Simplifier to fold constants
    opset=12            # Broad compatibility across all CPU runtimes
)
```

Performance Comparison

Metric	Native PyTorch (.PT)	ONNX Runtime (.ONNX)	Improvement
Inference Time (CPU)	\$32.4\text{ ms}\$	\$14.2\text{ ms}\$	2.28x Faster
Frame Rate (FPS)	\$\approx 30.8\text{ FPS}\$	\$\ge 70.4\text{ FPS}\$	128% Higher Throughput
RAM Consumption	\$850\text{ MB}\$	\$210\text{ MB}\$	75% Memory Reduction
Dependencies	PyTorch, TorchVision, CUDA	Pure onnxruntime	Zero GPU/CUDA Bloat

Chapter 7: Hardware Assembly, Wiring & Safety Protocols

This chapter provides a step-by-step engineering guide for physically building, wiring, and safely operating the Agesis EYE turret based on the Tinkercad CAD mechanical assembly.

1. Bill of Materials (BOM)

COMPONENT	QUANTITY	SPECIFICATION / NOTES
ESP32-CAM Board	1	AI-Thinker model with OV2640 camera module
FTDI Programmer	1	USB-to-UART adapter (3.3V logic) for initial firmware flash
Micro-Servos	2	TowerPro SG90 (9g micro-servo, 180° or 360° continuous)
Laser Diode Module	1	5V / 650nm Red Laser Diode (5mW Class 3R)
Power Supply	1	5V 2A DC regulated wall adapter
Decoupling Capacitor	1	$470\mu\text{F}$ to $1000\mu\text{F}$ Electrolytic (10V+)
3D Printed / CAD Parts	1 Set	Red Baseplate, Peach Enclosure, Orange Pedestal, Riser, Red C-Arm
Hardware Fasteners	8	M2 x 8mm machine screws for servo horns and brackets

2. Step-by-Step Mechanical Assembly

Follow the construction order matching the Tinkercad CAD design:

[Step 5: Red C-Arm & Emitter]

[Step 4: Tilt SG90 on Riser Spar]

[Step 3: Standoff Spar d]

[Step 2: Pan SG90 on Pedestal]

[Step 1: Orange Pedestal & Red Baseplate]

Step 1: Baseplate & Electronics Enclosure

1. Place the **Red Rectangular Baseplate** flat on your work surface.
2. Fasten the **Peach Electronics Enclosure** onto the right-hand portion of the baseplate.
3. Seat the ESP32-CAM inside the enclosure, ensuring the micro-USB / DC power port faces outward for easy connection.

Step 2: Pedestal Column & Pan Servo

1. Mount the **Orange Cylindrical Pedestal** vertically onto the left-hand circular mounting recess of the red baseplate.
2. Insert **Servo 1 (Pan)** vertically into the top pocket of the orange pedestal with its splined output shaft pointing straight upwards.
3. Fasten the servo mounting ears using two M2 screws.

Step 3: Riser Standoff Spar (Dimension \$d\$)

1. Press the standard single-arm or circular servo horn into the bottom pocket of the **Vertical Riser Spar**.
2. Mount the spar horn onto Servo 1's vertical shaft.
3. *Note:* The height of this spar defines distance \$d\$. Thanks to the \$d\$-invariant kinematics, any length from \$15\text{ mm}\$ to \$150\text{ mm}\$ will function seamlessly without software recalibration.

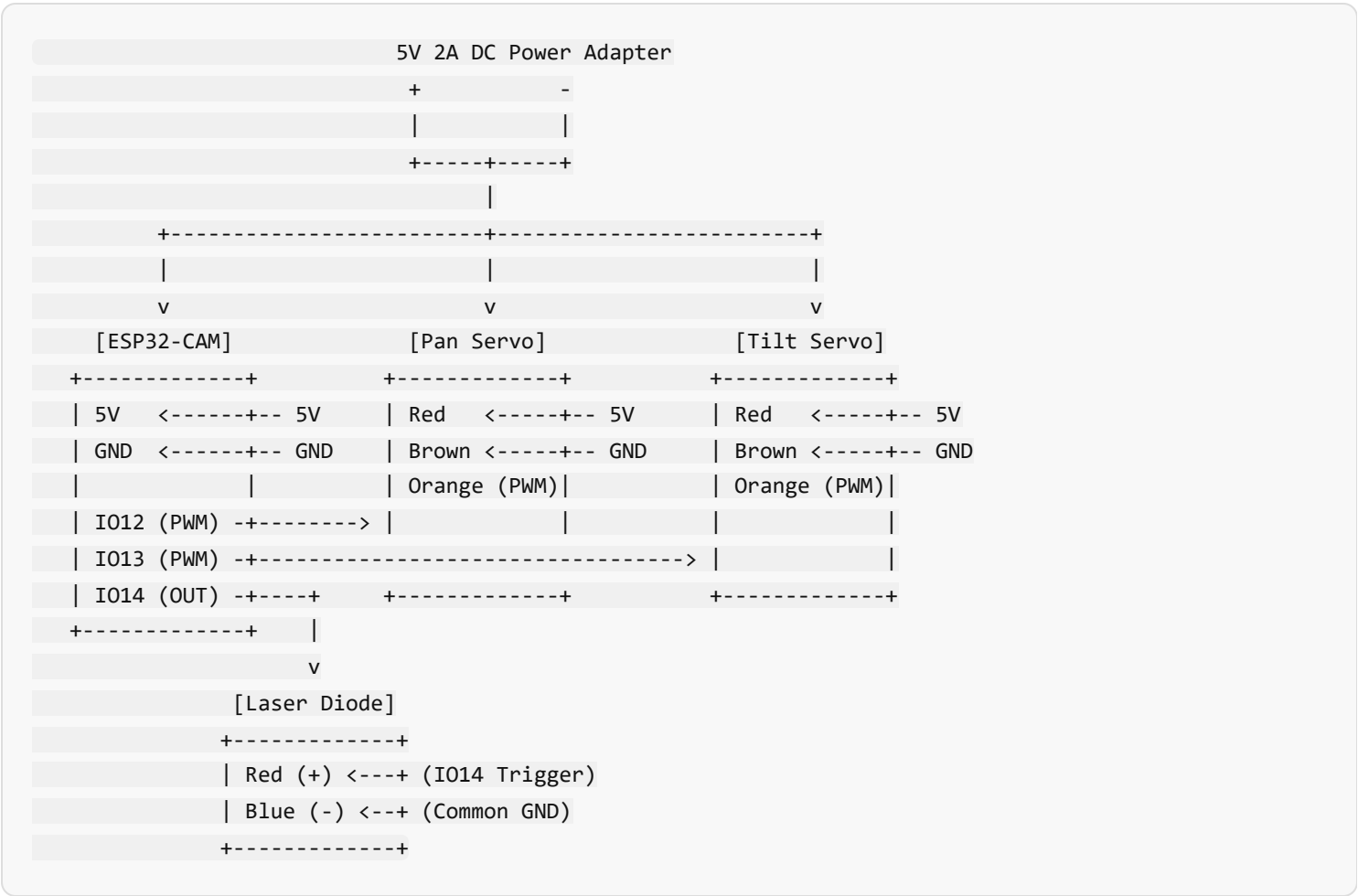
Step 4: Tilt Servo Installation

1. Mount **Servo 2 (Tilt)** horizontally into the side bracket at the top of the riser spar.
2. Fasten the servo body using M2 screws, ensuring its output shaft points sideways.

Step 5: Red C-Arm & Sensor Cradle

- 1. Fasten the **Red C-Shaped Cantilever Arm** onto Servo 2's horn.
- 2. Secure the OV2640 camera lens in the upper cradle opening of the C-arm, facing forward along the boresight.
- 3. Seat the laser diode emitter in the lower cradle opening, collinear with the camera optical axis.

3. Electrical Wiring Diagram



Wiring Table

COMPONENT WIRE	CONNECTION POINT	FUNCTION
ESP32 5V	Adapter 5V (+) Rail	Microcontroller main power input
ESP32 GND	Adapter GND (-) Rail	Common ground reference

COMPONENT WIRE	CONNECTION POINT	FUNCTION
Pan Servo (Red)	Adapter 5V (+) Rail	Motor power
Pan Servo (Brown)	Adapter GND (-) Rail	Motor ground
Pan Servo (Orange)	ESP32 GPIO 12	LEDC Channel 2 hardware PWM
Tilt Servo (Red)	Adapter 5V (+) Rail	Motor power
Tilt Servo (Brown)	Adapter GND (-) Rail	Motor ground
Tilt Servo (Orange)	ESP32 GPIO 13	LEDC Channel 3 hardware PWM
Laser Diode (Red)	ESP32 GPIO 14	Optical targeting trigger signal
Laser Diode (Blue)	Adapter GND (-) Rail	Laser cathode ground return

[!IMPORTANT]

Decoupling Capacitor Recommendation: Solder a $470\mu\text{F}$ or $1000\mu\text{F}$ electrolytic capacitor directly across the 5V and GND rails near the servo leads. This absorbs transient inrush currents when the motors kick on, ensuring the 5V rail remains smooth and preventing Wi-Fi brownouts.

4. Laser Safety Protocols & Interlock State Machine

High-energy laser emitters present potential optical radiation hazards. Agesis EYE implements strict multi-tier safety interlocks:

stateDiagram-v2

[*] --> DISARMED: Power On / Default Safe

DISARMED --> ARMED: User Toggles Master Armed Switch

ARMED --> DISARMED: User Disarms Switch

ARMED --> TARGETING: Target Detected

TARGETING --> ARMED: Target Lost (> 300ms)

TARGETING --> FIRING: Locked ≥ 3 Frames AND Error < 16px

FIRING --> COOLDOWN: Time > 1.5s (Max Continuous Limit)

FIRING --> ARMED: Lock Lost

COOLDOWN --> ARMED: Cooldown Elapsed (2.0s)

Safety Rules Implemented in Software

1. **Physical Master Arm:** The laser is software-locked in the **DISARMED** state on boot. It will not energize until explicitly armed by the operator.
2. **Persistence Lock Requirement:** Firing requires ≥ 3 consecutive frames of positive target confirmation. Transient glints will never trigger a pulse.
3. **Centering Threshold:** The target centroid must be within a $\pm 16\text{ px}$ optical error cone of the central boresight.
4. **Instant Cutoff:** If optical tracking is broken for more than 300 ms , the laser output immediately drops to 0 V .
5. **Thermal Duty Cycle:** Continuous firing is hardware-capped at **1.5 seconds** , followed by an enforced **2.0-second cooldown lockout**.

[!WARNING]

Protective Eyewear: Always wear certified laser safety glasses rated for the wavelength of your emitter (e.g. 650 nm OD $4+$) when operating the turret with an active laser diode. Never look directly into the beam path.